HW1

AI:

用java實作一個「骰子模擬器」，檔名HW1.java。
規格：
視窗標題「骰子模擬器」，尺寸 400×320，關閉時結束程式，開啟時置中

這是一個符合你所有規格要求，使用 Java Swing 開發的「骰子模擬器」程式碼。

請將以下程式碼存成 `HW1.java`：

Java

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.Random;

public class HW1 extends JFrame {

    private int rollCount = 0;
    private int rollSum = 0;

    private JLabel topLabel;
    private JLabel centerLabel;
    private JButton rollButton;
    private Random random;

    public HW1() {
        // 1. 視窗基本設定
        setTitle("骰子模擬器");
        setSize(400, 320);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // 關閉時結束程式
        setLocationRelativeTo(null); // 開啟時置中

        random = new Random();

        // 設定 Layout
        setLayout(new BorderLayout());

        // 2. 上方統計標籤
        topLabel = new JLabel("已擲 0 次，總和 0，平均 0.00", SwingConstants.CENTER);
        topLabel.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 18));
        topLabel.setBorder(BorderFactory.createEmptyBorder(15, 0, 10, 0));
        add(topLabel, BorderLayout.NORTH);
```

```
        // 3. 中央點數標籤 (字體 60pt)
        centerLabel = new JLabel("?", SwingConstants.CENTER);
        centerLabel.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 60));
        add(centerLabel, BorderLayout.CENTER);

        // 4. 下方按鈕
        rollButton = new JButton("擲骰子");
        rollButton.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 20));
        rollButton.setPreferredSize(new Dimension(0, 60));

        // 按鈕點擊事件
        rollButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                rollDice();
            }
        });
        add(rollButton, BorderLayout.SOUTH);
    }
```

```
    private void rollDice() {
        // 隨機產生 1-6
        int roll = random.nextInt(6) + 1;

        // 更新統計數據
        rollCount++;
        rollSum += roll;
        double average = (double) rollSum / rollCount;

        // 更新上方文字 (X.XX 格式)
        topLabel.setText(String.format("已擲 %d 次，總和 %d，平均 %.2f", rollCount, rol

        // 更新中央點數
        centerLabel.setText(String.valueOf(roll));

        // 判斷顏色：6 為綠色(使用較深的綠色確保清晰度)，1 為紅色，其餘黑色
        if (roll == 6) {
            centerLabel.setForeground(new Color(0, 180, 0));
        } else if (roll == 1) {
            centerLabel.setForeground(Color.RED);
        } else {
            centerLabel.setForeground(Color.BLACK);
        }
    }

    public static void main(String[] args) {
        // 確保 GUI 在 Event Dispatch Thread (EDT) 執行，這是 Swing 的安全規範
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                new HW1().setVisible(true);
            }
        });
    }
}
```

Reflection: Nothing to change

My Thoughts:

1. random.nextInt(6) + 1 這個寫法的原理。因為 nextInt(6) 會產生 0 到 5 的數字，所以加上 1 剛好完美模擬了現實中骰子 1 到 6 點的範圍。
2. BorderLayout 的排版方式，將上方的統計數據放在 NORTH、中央

的骰子結果放在 CENTER、下方的按鈕放在 SOUTH，這種直觀的區塊劃分方式比起絕對座標排版，更容易做出整齊的介面

HW2

AI



```java
public HW2() {
    // 設定視窗基本屬性
    setTitle("單位換算器");
    setSize(480, 280);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLayout(new BorderLayout());

    // ===== NORTH 區域 =====
    JPanel northPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 20));
    northPanel.add(new JLabel("換算類型："));
    typeComboBox = new JComboBox<>(types);
    northPanel.add(typeComboBox);
    add(northPanel, BorderLayout.NORTH);

    // ===== CENTER 區域 =====
    // 使用 GridLayout(2, 1) 來放置兩列
    JPanel centerPanel = new JPanel(new GridLayout(2, 1));

    // 第一列：來源輸入框與單位
    JPanel row1 = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 10));
    inputField = new JTextField(15);
    sourceUnitComboBox = new JComboBox<>();
    row1.add(inputField);
    row1.add(sourceUnitComboBox);

    // 第二列：目標輸出框與單位
    JPanel row2 = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 10));
    outputField = new JTextField(15);
    outputField.setEditable(false); // 設定為不可編輯，純顯示結果
    targetUnitComboBox = new JComboBox<>();
    row2.add(outputField);
    row2.add(targetUnitComboBox);

    centerPanel.add(row1);
    centerPanel.add(row2);
    add(centerPanel, BorderLayout.CENTER);

    // ===== SOUTH 區域 =====
    JPanel southPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 20));
    convertButton = new JButton("換算");
    southPanel.add(convertButton);
    add(southPanel, BorderLayout.SOUTH);
```

```
        // ===== 事件監聽設定 =====
        // 當換算類型改變時，更新下拉選單內容
        typeComboBox.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                updateUnitComboBoxes();
            }
        });

        // 點擊換算按鈕進行計算
        convertButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                performConversion();
            }
        });

        // 初始化第一次的單位下拉選單內容
        updateUnitComboBoxes();
    }

    // 更新單位下拉選單的副程式
    private void updateUnitComboBoxes() {
        int selectedType = typeComboBox.getSelectedIndex();

        // 清除舊選項
        sourceUnitComboBox.removeAllItems();
        targetUnitComboBox.removeAllItems();

        String[] currentUnits = null;
        if (selectedType == 0) {
            currentUnits = lengthUnits;
        } else if (selectedType == 1) {
            currentUnits = weightUnits;
        } else if (selectedType == 2) {
            currentUnits = tempUnits;
        }

        // 加入新選項
        if (currentUnits != null) {
            for (String unit : currentUnits) {
                sourceUnitComboBox.addItem(unit);
                targetUnitComboBox.addItem(unit);
            }
        }
```

```
    // 執行換算邏輯的副程式
    private void performConversion() {
        try {
            String inputText = inputField.getText().trim();
            if (inputText.isEmpty()) return;

            double inputValue = Double.parseDouble(inputText);
            int typeIndex = typeComboBox.getSelectedIndex();
            int sourceIndex = sourceUnitComboBox.getSelectedIndex();
            int targetIndex = targetUnitComboBox.getSelectedIndex();

            if (sourceIndex == -1 || targetIndex == -1) return;

            double result = 0.0;

            if (typeIndex == 0) {
                // 【長度換算】先統一轉為公尺，再轉為目標單位
                double inMeters = 0;
                switch (sourceIndex) {
                    case 0: inMeters = inputValue; break;                // 公尺
                    case 1: inMeters = inputValue * 0.01; break;         // 公分
                    case 2: inMeters = inputValue * 0.0254; break;       // 英吋
                    case 3: inMeters = inputValue * 0.3048; break;       // 英尺
                }
                switch (targetIndex) {
                    case 0: result = inMeters; break;
                    case 1: result = inMeters / 0.01; break;
                    case 2: result = inMeters / 0.0254; break;
                    case 3: result = inMeters / 0.3048; break;
                }
            } else if (typeIndex == 1) {
                // 【重量換算】先統一轉為公斤，再轉為目標單位
                double inKg = 0;
                switch (sourceIndex) {
                    case 0: inKg = inputValue; break;                    // 公斤
                    case 1: inKg = inputValue * 0.001; break;            // 公克
                    case 2: inKg = inputValue * 0.45359237; break;       // 磅
                    case 3: inKg = inputValue * 0.0283495231; break;     // 盎司
                }
                switch (targetIndex) {
                    case 0: result = inKg; break;
                    case 1: result = inKg / 0.001; break;
                    case 2: result = inKg / 0.45359237; break;
                    case 3: result = inKg / 0.0283495231; break;
                }
```

```
            } else if (typeIndex == 2) {
                // 【溫度換算】先統一轉為攝氏，再轉為目標單位
                double inCelsius = 0;
                switch (sourceIndex) {
                    case 0: inCelsius = inputValue; break;                    //
                    case 1: inCelsius = (inputValue - 32) * 5.0 / 9.0; break;  //
                    case 2: inCelsius = inputValue - 273.15; break;           //
                }
                switch (targetIndex) {
                    case 0: result = inCelsius; break;
                    case 1: result = inCelsius * 9.0 / 5.0 + 32; break;
                    case 2: result = inCelsius + 273.15; break;
                }
            }

            // 將結果顯示於輸出框，預設顯示小數點後 4 位
            outputField.setText(String.format("%.4f", result));

        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "請在上方輸入框填入有效的數字", "格式錯誤"
        }
    }

    public static void main(String[] args) {
        // 使用事件分派執行緒(EDT)來啟動 GUI
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                HW2 frame = new HW2();
                // 置中顯示
                frame.setLocationRelativeTo(null);
                frame.setVisible(true);
            }
        });
    }
}
```

Reflection: Nothing to change

My Thoughts:

1. 程式碼採用了「先轉成基準單位，再轉成目標單位」的策略。將所有長度先轉為公尺（inMeters），再依據目標單位的索引值（targetIndex）轉換為最終結果。
2. 例外處理:透過 try-catch 捕捉 NumberFormatException，當輸入空白或英文字母時，會彈出 JOptionPane 警告視窗而不是直接讓程式崩潰閃退